

TITLE:

INTRODUCTION TO PROLOG

OBJECTIVES:

- To gain an overview of PROLOG programming.
- To learn the basics of PROLOG programming, implement some logic, and analyze the output.

THEORY:

PROLOG (Programming in Logic) is a high-level declarative programming language primarily used in the fields of Artificial Intelligence (AI), Natural Language Processing (NLP), and knowledge-based systems. Unlike procedural languages such as C or Java, PROLOG focuses on *what* the problem is rather than *how* to solve it. It is based on formal logic and consists of facts, rules, and queries. The programmer defines relationships and logical conditions, and the PROLOG interpreter automatically determines the solution by applying logical inference and backtracking.

BASIC COMPONENTS OF PROLOG:

- Facts represent basic assertions about the problem domain and state relationships that are always True.
For Example : `parent(john, mary).` % This means John is the parent of Mary.
- Rules define conditional relationships between facts. A rule consists of a head and a body, connected using `":-"`. which can be read as "if".
For example: `grandparent(X, Y) :- parent(X, Z), parent(Z, Y).`
% This means X is the grandparent of Y if X is the parent of Z and Z is the parent of Y. Queries:
- Queries are used to ask questions to the PROLOG system based on the defined facts and rules.
For Example : `parent(john, mary).`
% PROLOG responds with True or False, or provides values for variables.

PROGRAM CODE:

1. WAP to display "Hello World" using SWI prolog
`hello:-format('Hello World').`
`.`

OUTPUT:

```
?- hello (Press enter)
Hello World
True.
```

2. Solve the following problems using SWI prolog as per the goals given.
% Schedule facts
`schedule(tuesday, math).`
`schedule(tuesday, english).`
`schedule(wednesday, programming).`
`schedule(wednesday, spanish).`
`schedule(thursday, circuits).`
`schedule(friday, none).`

% Difficulty facts
`difficulty(programming, hard).`
`difficulty(math, hard).`
`difficulty(english, easy).`
`difficulty(spanish, medium).`
`difficulty(circuits, hard).`

```
% Class information rule
classinformation(Day, Class, Diff) :-
    schedule(Day, Class).
difficulty(Class, Diff).
```

OUTPUT:

```
?- schedule(monday, programming).
False
```

```
?- schedule(tuesday, english).
True.
```

```
?- difficulty(programming, easy).
False
```

```
?- difficulty(programming, hard).
True.
```

```
?- classinformation(tuesday, Class, Diff).
Class = math,
Diff = hard ;
Class = english,
Diff = easy.
```

```
?- classinformation(tuesday, Class, easy).
Class = english.
```

```
?- classinformation(Day, Class, hard).
Day = tuesday,
Class = math ;
Day = wednesday,
Class = programming ;
Day = thursday,
Class = circuits.
```

```
?- classinformation(Day, english, hard).
False
```

```
?- classinformation(Day, english, easy).
Day = tuesday.
```

3. Solve the following problems using SWI prolog as per the goals given.

```
% Cuisine facts
cuisine(italian, marios).
cuisine(italian, pasta_paradise).
cuisine(mexican, taco_fiesta).
cuisine(mexican, chili_beans).
cuisine(indian, spice_route).
cuisine(japanese, sushi_zen).
cuisine(japanese, wasabi_world).
cuisine(burger, burger_kingdom).
```

```
% Price facts
price(marios, expensive).
price(pasta_paradise, moderate).
price(taco_fiesta, cheap).
price(chili_beans, moderate).
price(spice_route, expensive).
price(sushi_zen, expensive).
price(wasabi_world, moderate).
price(burger_kingdom, cheap).
```

```
% Location facts
location(marios, downtown).
location(pasta_paradise, uptown).
location(taco_fiesta, downtown).
location(chili_beans, midtown).
location(spice_route, downtown).
location(sushi_zen, uptown).
location(wasabi_world, downtown).
location(burger_kingdom, midtown).
```

```
% Rating facts
rating(marios, 4).
rating(pasta_paradise, 3).
rating(taco_fiesta, 5).
rating(chili_beans, 4).
rating(spice_route, 5).
rating(sushi_zen, 4).
rating(wasabi_world, 3).
rating(burger_kingdom, 4).
```

```
% Combined restaurant information rule
restaurantinfo(Name, Cuisine, Price, Area, Stars) :-
    cuisine(Cuisine, Name),
    price(Name, Price),
    location(Name, Area),
    rating(Name, Stars).
```

OUTPUT:

```
?- cuisine(italian, marios).
True.
```

```
?- cuisine(chinese, spice_route).
False.
```

```
?- price(sushi_zen, expensive).
True.
```

```
?- price(taco_fiesta, expensive).
False.
```

```
?- location(pasta_paradise, downtown).
False.
```

?- cuisine(italian, Restaurant).
Restaurant = marios ;
Restaurant = pasta_paradise.

?- price(Restaurant, cheap).
Restaurant = taco_fiesta ;
Restaurant = burger_kingdom.

?- location(Restaurant, downtown).
Restaurant = marios ;
Restaurant = taco_fiesta ;
Restaurant = spice_route ;
Restaurant = wasabi_world.

?- rating(Restaurant, 5).
Restaurant = taco_fiesta ;
Restaurant = spice_route.

?- restaurantinfo(Name, mexican, Price, Area, Stars).
Name = taco_fiesta,
Price = cheap,
Area = downtown,
Stars = 5 ;
Name = chili_beans,
Price = moderate,
Area = midtown,
Stars = 4.

?- restaurantinfo(Name, Cuisine, expensive, downtown, Stars).
Name = marios,
Cuisine = italian,
Stars = 4 ;
Name = spice_route,
Cuisine = indian,
Stars = 5 ;
False.

?- restaurantinfo(Name, japanese, Price, Area, 4).
Name = sushi_zen,
Price = expensive,
Area = uptown ;
False.

?- restaurantinfo(taco_fiesta, Cuisine, Price, Area, Stars).
Cuisine = mexican,
Price = cheap,
Area = downtown,
Stars = 5.

?- cuisine(Cuisine, Name), price(Name, cheap).
Cuisine = mexican,
Name = taco_fiesta ;

Cuisine = burger,
Name = burger_kingdom.

?- restaurantinfo(Name, Cuisine, moderate, downtown, Rating), Rating > 3.
False.

?- location(Name, uptown), rating(Name, Stars), Stars >= 4.
Name = sushi_zen,
Stars = 4.

?- restaurantinfo(Name, italian, Price, Area, Stars), (Price = cheap ; Price = moderate).
Name = pasta_paradise,
Price = moderate,
Area = uptown,
Stars = 3.

?- restaurantinfo(Name, _, expensive, _, 5).
Name = spice_route ;
False.

?- restaurantinfo(Name, Cuisine, _, downtown, _), Cuisine = italian.
Name = taco_fiesta,
Cuisine = mexican ;
Name = spice_route,
Cuisine = indian ;
Name = wasabi_world,
Cuisine = japanese ;
False.

?- restaurantinfo(_, mexican, cheap, _, 4).
False.

DISCUSSION AND CONCLUSION:

In this lab, we represented various types of information using Prolog facts and retrieved combined data through rules. This exercise helped us understand how Prolog manages logical relationships and querying. It demonstrated how logic can be structured and applied effectively in AI and other logic-based programming tasks. Through this lab, we learned how facts, rules, and queries work together to solve problems and generate meaningful outputs.



LAB NO : 01

LAB ON : INTRODUCTION TO PROLOG

SUMMITTED BY:

Name: Oshan Bajracharya

Roll No: 230328

Faculty: BE-Computer

Group: B

SUMMITTED TO:

Department Of ICT

Date of Experiment:

Date of Submission:

